课程编号：A0801053010

分布式系统导论课程设计

雾桌（MistTable）多人在线虚拟桌游平台的技术分析和概要设计



组名	完成作业组	选题方向	B：多人实时协作的在线虚拟桌游平台
组长(100%)姓名	周宇斌	组长学号	20246085
副组长(98%)姓名	刘润七	副组长学号	20246162
副组长(98%)姓名	刘万轩	副组长学号	20246063
副组长(98%)姓名	周星宇	副组长学号	20245986
组员(80%)姓名和学号(不多于 4 人)	无		
评定成绩		评定人	
		评定日期	

东北大学软件学院

评分标准	得分
需求明确：清晰定义分布式系统的技术需求(25%)；	25
设计完整：涵盖大部分所学知识点(15%)；	15
设计正确：可以完成既定技术需求(30%)；	30
自主知识运用：是自己想的还是查阅资料获得的(20%)；	20
选题新颖：常规的教科书均可见的还是较为新颖的分布式系统(10%)；	10
大段文献抄袭且不给出引用的，对应考核点按零分处理	

目　录

1. 系统简介 4
   1.1 系统背景与业务定位 4
   1.2 为什么属于分布式系统 4
   1.3 核心业务功能与角色权限 4
   1.4 分布式技术需求与设计假设 5
   1.5 课件理论联系：分布式系统的目标、挑战与事务边界 6
2. 体系结构 7
   2.1 技术需求 7
   2.2 概要设计 7
      (1) 客户端—服务器分层协作架构 7
      (2) 房间级单权威处理路径与扩展方式 8
      (3) 核心架构选型对比：C/S 架构与全 P2P 架构的权衡 8
3. 进程 9
   3.1 技术需求 9
   3.2 概要设计 9
      (1) 客户端与服务端进程职责分工 9
      (2) 多房间分配与并行处理方式 9
      (3) 课件理论分析：并发模型、虚拟化与代码迁移适用性 10
4. 通信 10
   4.1 技术需求 10
   4.2 概要设计 11
      (1) 请求—应答与实时协作的通信分工 11
      (2) 角色标记移动与状态传播时序 11
      (3) 课件理论分析：瞬态通信 vs 持久通信、消息广播与恢复边界 12
5. 命名 13
   5.1 技术需求 13
   5.2 概要设计 13
      (1) 稳定实体标识设计 13

(2) 邀请码凭证与房间定位机制 13
(3) 课件理论分析：平面命名、结构化命名与 DHT/Chord 方案对比 14
6. 同步 15
6.1 技术需求 15
6.2 概要设计 15
(1) 基于房间权威定序与对象版本的冲突处理机制 15
(2) 掷骰与状态判定的有序性保障 16
(3) 课件理论分析：物理时钟偏差、逻辑时钟与并发控制算法比较 16
7. 一致性与复制 17
7.1 技术需求 17
7.2 概要设计 17
(1) 业务分级一致性模型构建 17
(2) 战争迷雾与成员权限的状态投影隔离 18
(3) 课件理论分析：主备复制与 Quorum 仲裁机制对比 19
8. 容错性与故障恢复 20
8.1 技术需求 20
8.2 概要设计 20
(1) 网络超时分析与基于操作唯一标识的幂等重试机制 20
(2) 客户端断线重连与增量/全量快照差分恢复流程 20
(3) 协作服务进程故障接管与恢复前提边界 22
(4) 课件理论分析：故障分类模型、可靠通信与 2PC/3PC 适用边界 22
9. 总结 23
9.1 系统设计与分布式知识点对应回顾 23
9.2 核心设计取舍与工程理性总结 24
9.3 设计局限性与未验证假设声明 24
参考文献 25
附录 (得分栏勿填) 26

1. 系统简介

1.1 系统背景与业务定位

在计算机支持的协同工作（CSCW）与线上互动娱乐领域，桌面角色扮演游戏（Tabletop Role-Playing Games, TTRPG，如跑团等）正在由线下实体聚会转向通过网络虚拟桌面（Virtual Tabletop, VTT）展开协同。参考优秀的轻量化虚拟桌游实践（如 Owlbear Rodeo [9]），本课程设计提出“雾桌（MistTable）”多人实时协作在线虚拟桌游平台。雾桌旨在通过纯浏览器终端，为主持人（Host）与玩家（Player）提供一个支持多人在线协同查看场景、移动角色标记（Token）、投掷骰子并由主持人控制战争迷雾（Fog of War）探索视野的在线跑团环境。

本概要设计报告的核心任务，并非实现一个具备支付、音视频或复杂商业规则系统的完整商业平台，而是围绕分布式系统的核心理论，系统阐述身处不同网络节点的多个用户如何共同操作同一个虚拟桌游房间，并在网络延迟、并发冲突以及局部节点异常等条件下，依然获得可解释、可恢复的共享状态。报告将重点结合东北大学软件学院《分布式系统导论》课程教学大纲与宋杰老师课件（Chapter 1–8）[1] 展开推演。

1.2 为什么属于分布式系统

根据分布式系统经典定义（Tanenbaum 著作 [2] 及课程课件 Chapter 1 [1]），分布式系统是由通过网络连接的多个自治计算机进程组成的集合，对用户表现为一个统一的连贯系统。雾桌平台之所以属于典型的分布式系统，体现在以下几个根本特征：

（1）独立物理进程协作：系统由分布在不同地理位置的浏览器客户端进程、接入服务进程、房间协作服务进程以及后端持久化存储节点组成。各个节点运行在各自独立的物理设备之上，完全通过网络消息传递实现协同，不存在全局物理共享内存。

（2）网络不可靠性与非零延迟：节点之间的交互依托公共互联网，必然面临网络传输延迟、网络抖动、报文无序到达或丢失等现象。系统无法假定任何跨网络操作能够瞬间完成或绝对可靠。

（3）缺乏统一的全局物理时钟：各个玩家设备与服务器节点的本地物理时钟存在不可避免的时间偏差，系统无法直接以客户端本地时间戳作为判定操作先后的权威依据。

（4）并发操作与共享状态维护：多名玩家可能在极短时间内对同一标记或场景要素发起移动或修改请求，系统需要在分布式环境下定义合理的操作处理顺序，防止状态产生歧义或覆盖。

（5）独立局部故障（Partial Failure）：网络中个别客户端意外掉线或某服务节点异常，属于典型的局部故障。系统必须在局部节点异常时保障其余节点的正常交互，并为异常节点恢复提供可靠依据。

1.3 核心业务功能与角色权限

系统定义两类核心用户角色：主持人（Host）与玩家（Player）。主持人负责创建房间、选择地图、调整战争迷雾与场景状态，并控制成员权限；玩家加入房间后，可查看经授权可见的场景、移动所绑定的自身角色标记并进行掷骰检定。系统核心业务功能与分布式技术问题的映射如表 1-1 所示。

表 1-1 系统核心业务功能与分布式技术问题映射表

功能编号	业务功能名称	用户行为描述	对应的分布式系统技术问题
FR-01	房间生命周期管理	主持人创建房间、配置属性；玩家通过凭据加入或退出	逻辑命名、服务定位与状态初始化
FR-02	预置地图场景呈现	主持人选定预置地图，成员按权限加载地图瓦片	静态资源命名、受控瓦片访问、共享场景初始化
FR-03	角色标记实时协作	获授权成员拖拽角色标记并同步位置坐标	并发请求处理、操作权威定序、增量状态同步
FR-04	战争迷雾权限隔离	主持人控制可见区域，遮蔽未探索内容	服务端受限投影、视野访问控制与信息隔离
FR-05	掷骰检定与结果同步	玩家请求掷骰，系统生成结果并在房间成员间同步	权威随机生成、请求幂等处理、有序事件广播
FR-06	增量状态实时分发	已确认的状态变化及时向房间在线成员推送	实时通信通道、面向群组的消息广播
FR-07	断线重连与状态恢复	客户端因网络抖动离线后重连，恢复房间状态	断线重新认证、增量补发与当前状态快照恢复

1.4 分布式技术需求与设计假设

系统明确界定功能范围：系统仅支持主持人选择系统预置的标准地图资源，不包含用户上传自定义地图文件、格式转码及外部存储流程。系统暂不纳入语音视频通话、支付系统、第三方插件市场、复杂规则引擎、全网状对等拓扑（P2P Mesh）及跨地域容灾设计，以确保概要设计聚焦于核心协同机制。

在系统规模与设计目标方面，确立了五项核心设计假设指标，如表 1-2 所示。

表 1-2 系统性能与规模设计假设指标表

指标分类	设计目标 / 假设数值	指标口径与测量边界说明	设计含义与约束
单房间承载规模	最多 8 名同时在线成员 (含主持人)	单房间实时通信并发连接数 ≤8	面向小型协作，避免无必要的超大规模群组通信复杂度
初始系统总体规模	约 100 个活跃房间并行处理	系统在设计推演中假定承载约 100 个自治桌游房间	为多房间分配与并行处理提供规模参考基准
实时状态更新延迟	P95 目标 ≤300 ms	同地域正常网络下，服务端权威提交至在线端收到更新	要求通信机制采用长连接双向通道，避免低效的定时轮询
常规管理 API 延迟	P95 目标 ≤500 ms	服务端接收管理请求至返回响应时间 (不含地图资源下载)	REST 接口面向低频控制操作，采用轻量无状态处理
网络断线恢复时间	授权状态恢复目标 ≤ 5 秒	普通规模房间在连接恢复后完成授权状态对齐的目标时间	要求服务端提供增量事件缓冲或快照差分恢复途径

【关键设计限定声明】: 上述表 1-2 中列出的各项指标数值，均为课程设计推演中的“设计假设与设计目标”，并非指导教师规定的硬性指标，亦未经真实系统的实现编码、压力测试或故障演练验证。报告在后续各章节的方案设计与分析中，均严格以此作为理论推导边界，不将其描述为已验证的实测性能结果。

1.5 课件理论联系：分布式系统的目标、挑战与事务边界

对照宋杰老师课件 Chapter 1（Introduction）关于分布式系统核心理论的论述 [1]，系统设计体现了如下原理映射：

1. 分布式系统目标与透明性权衡：资源共享使不同终端能够共同操作地图、标记与操作历史；访问透明性隐藏服务接口背后的实现差异，位置与迁移透明性使房间换节点后仍可用稳定标识访问；复制透明性隐藏存储副本的位置。故障透明性有成本，故障切换或恢复时应向用户说明暂时不可写。透明性讨论的是隐藏分布细节，不意味着让玩家看到无权访问的数据；迷雾属于访问控制。开放性依靠标准接口，可扩展性通过房间间并行、异步通信和静态资源分发获得。[1]

2. 分布式环境的设计挑战：课件强调网络并非完全可靠、延迟并非为零。系统设计将网络丢包、延迟抖动与偶发断线视为系统必须面对的前提，不在逻辑层假定网络消息具有即时性与绝对可靠性。

3. 事务概念与持久化边界界定：课件 Chapter 1 详细分析了事务的 ACID 特性 [1, 10]。本设计将单次操作引发的状态变更、房间递增序号及操作记录纳入单个持久化存储事务中提交。在此必须明确界定：数据库或存储层事务提交成功，仅代表该操作在服务端持久化边界内完成了保存；它与网络广播消息是否已成功送达所有在线客户端，是两个不同层面的分布式问题，不能混为一谈。

2. 体系结构

2.1 技术需求

在虚拟桌游多端协同环境中，体系结构设计必须能够支持实时状态同步、权限校验与多房间并发处理。核心技术需求包括：

（1）共享状态权威判定需求：桌游协同涉及角色位置、场景切换及投骰记录，系统需要对同一房间内被接受的合法操作给出一致、明确的处理顺序，避免多端状态出现歧义。

（2）主持人权限与信息隔离需求：TRPG 跑团依赖战争迷雾机制遮蔽未探索区域。隐藏在迷雾后的要素信息需在架构层面进行受限管理，防范客户端直接读取完整底层数据造成的透视作弊。

（3）多房间并行自治与扩展需求：在面对约 100 个活跃房间的设计假设下，不同房间的操作数据完全独立，架构应天然支持房间间的并行隔离，允许通过水平扩展服务能力满足规模需求。

2.2 概要设计

(1) 客户端—服务器分层协作架构

系统采用分层客户端—服务器（Layered Client-Server）架构。[1, 7] 在此基础上，本设计用事件通知传播房间状态。系统整体划分为客户端层、接入层、应用与房间协作服务层、持久化与基础设施层四个逻辑层次。系统总体逻辑架构如图 1 所示。

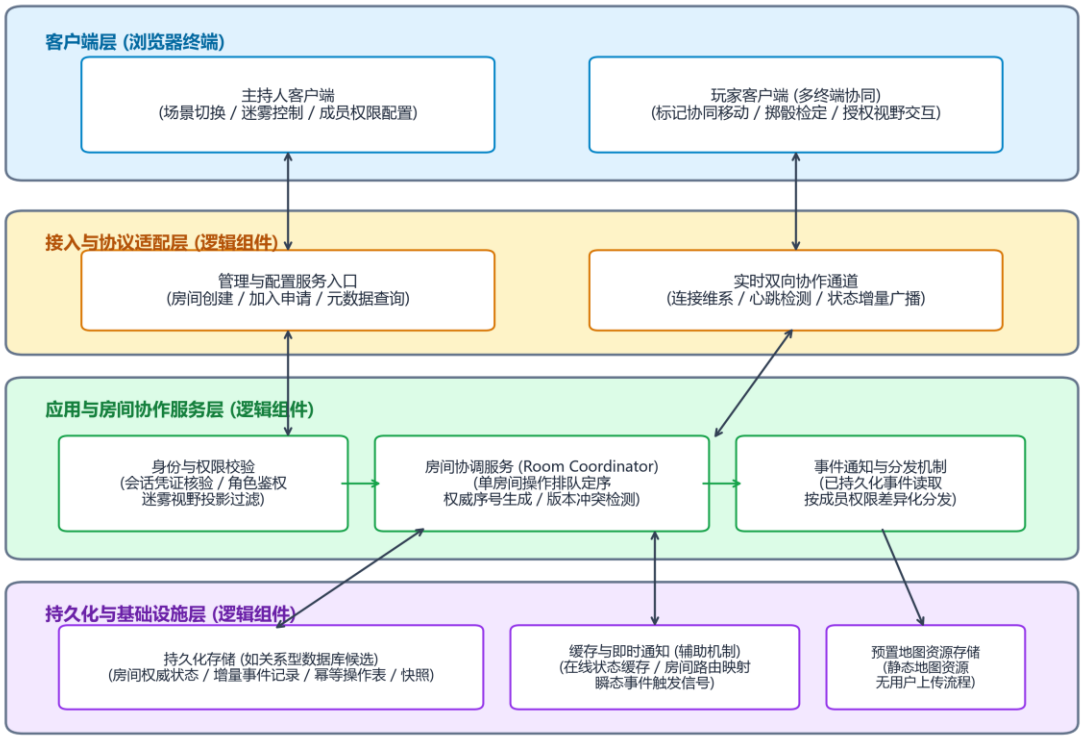


图 1　系统总体逻辑架构图

结合图 1，系统各逻辑组件的职责与调用边界如下：

1. 客户端层（Browser Clients）：包含主持人客户端与玩家客户端，运行于现代 Web 浏

览器。负责渲染地图与图元、捕获用户交互、提供非权威的本地拖拽视觉预览，并通过网络向服务端提交操作请求。

2. 接入与协议适配层（Access Layer）：负责外部网络连接管理。包含处理常规房间管理与元数据查询的管理接口服务，以及承载长连接全双工通信的实时协作网关，承担身份凭证校验与连接保活工作。

3. 应用与房间协作服务层（Room Collaboration Service Tier）：系统的核心协作中枢。包含身份权限校验、房间路由及房间协作服务（Room Coordinator）。逻辑上，每个房间在任一时刻由单一权威路径负责排队处理操作、校验规则、检测冲突并生成递增事件序号；同时运行视野过滤逻辑，将房间内部状态转换为不同角色的可见视图。

4. 持久化与基础设施层（Storage & Infrastructure Tier）：权威数据库保存房间状态、增量事件及幂等结果；路由缓存和在线状态属于可重建辅助信息。地图资源服务保管预置底图原文件，按会话角色及当前迷雾裁剪并返回授权瓦片，原文件不向玩家提供公开直链。公共 CDN 仅缓存公开、非敏感资源；含受限信息的瓦片须经鉴权且禁止共享缓存。

(2) 房间级单权威处理路径与扩展方式

为了兼顾单房间状态的一致性与全系统的处理能力，系统确立了‘不同房间并行处理，单房间权威定序’的设计思路：（1）房间间并行隔离：不同房间之间不存在数据依赖与共享状态，系统可按房间标识（roomId）将不同房间分配给不同的协作服务实例独立处理，支持通过横向扩容满足多房间负载；（2）单房间权威路径：对于特定房间内部的共享写操作，在逻辑上由一个权威处理路径顺序处理，避免了同一房间多个写入者之间的复杂分布式锁竞争，有助于保证操作顺序的清晰与可解释。

(3) 核心架构选型对比：C/S 架构与全 P2P 架构的权衡

在系统架构选型中，对等网络（Peer-to-Peer, P2P / WebRTC Mesh）常被用于实时协同。系统对分层 C/S 与 P2P 进行了对比分析，如表 2-1 所示。

表 2-1 客户端—服务器（C/S）与对等网络（P2P）架构方案对比表

评估维度	分层客户端—服务器架构 (C/S，当前方案)	对等网络架构 (P2P / Mesh，备选方案)
权威状态管理	服务端统一仲裁与落盘，规则校验与防作弊边界清晰	需另行设计定序与合并；CRDT [13] 仅保证满足相应条件的数据类型收敛
战争迷雾信息隔离	服务端依据权限计算视野差分投影，未探索数据在网络层被过滤	若全网分发底图依赖前端遮罩，存在客户端本地内存提取未探索要素的风险
网络连接拓扑	星型拓扑：客户端仅与服务端保持连接，单端连接开销较小	全网状网格：N 人房间需维护 N(N-1)/2 条连接，对弱网节点上行带宽压力大
断线状态恢复	服务端存储作为权威记录，掉线用户重连可从服务端恢复状态	房主或核心对等节点掉线可能造成状态丢失，选主与数据恢复协调复杂
架构取舍与代价	服务端承担网络与计算开销，是关键依赖，需设计容错与恢复机制	去中心化降低了中心服务器带宽负担，但协同逻辑与安全控制显著复杂

由表 2-1 可见，虽然 P2P 架构具备去中心化与节省服务端带宽的优点，但桌游平台具备显著的信息不对称性（战争迷雾）与单房间多用户强因果交互特征。P2P 架构在处理权限安全隔离、状态冲突收敛（如引入无冲突复制数据类型 CRDT [13]）与断线恢复方面需要

付出极高的算法与协调代价。分层 C/S 架构以明确的服务端权威较好地契合了业务需求，代价是服务端成为关键依赖，需在后续章节重点展开进程、通信与容错设计。

3. 进程

3.1 技术需求

在进程组织与并发处理层面，系统面临如下技术需求与约束：

（1）连接承载与并发处理需求：接入服务需承载多客户端并发长连接，必须采用高效的并发 I/O 模型，避免因客户端等待或网络慢速传输耗尽系统线程资源。

（2）前端交互响应与计算解耦：浏览器端需处理图形渲染与网络事件响应，UI 主线程应避免被大开销的本地计算阻塞，维持交互流畅性。

（3）多房间进程调度与隔离需求：单个房间内的操作处理与密集计算不应干扰其他独立房间的正常协同。

3.2 概要设计

(1) 客户端与服务端进程职责分工

系统确立了‘客户端负责交互与渲染展现，服务端负责规则核验与权威状态维护’的职责划分 [1, 2]。职责分工如表 3-1 所示。

表 3-1 客户端进程与服务端协作进程职责分工表

处理阶段	浏览器客户端进程职责	服务端协作进程职责
用户交互与指令发起	捕获用户拖拽输入，生成非权威视觉预览，构造操作请求发送	接收操作请求报文，进行初步格式检验，不直接信任前端坐标
权限校验与规则判定	基于本地已知状态做必要的前置防呆校验 (非安全防线)	权威核验会话身份、角色归属、移动距离合法性及迷雾阻挡
并发控制与版本判定	提交请求附带本地基准版本号 (base_version)	基于当前对象状态进行版本匹配检查，拒绝过期操作
状态确认与序号赋予	无权擅自落地共享状态，维持临时未确认状态	确认合法操作并更新状态，为有效事件分配房间自增序号
视野投影与事件下发	接收经权限过滤后的更新，更新本地画布，清理无效对象	根据当前战争迷雾多边形剪裁可见性，按角色分发状态事件

(2) 多房间分配与并行处理方式

针对约 100 个活跃房间的设计假设，多房间分配与并行处理示意如图 2 所示。

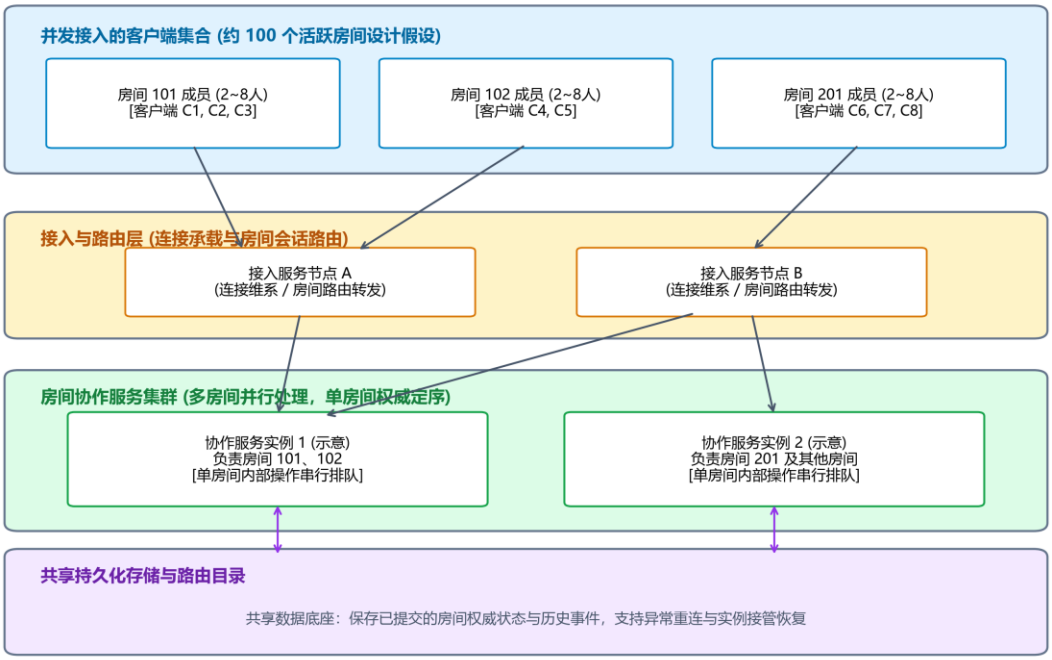


图 2　多房间分配与并行处理拓扑图

结合图 2，多房间处理调度逻辑如下：（1）无状态接入层转发：客户端连接接入服务节点后，接入层仅维护连接链路，不保存业务状态，可根据房间标识将会话请求路由给负责该房间的协作服务节点；（2）房间自治处理：不同的房间可分配给不同的协作服务实例或不同工作单元并行处理；在单个房间内部，操作请求按到达顺序由权威路径排队执行，保证同一房间内的状态推进具备可解释的明确顺序。

(3) 课件理论分析：并发模型、虚拟化与代码迁移适用性

对照宋杰老师课件 Chapter 3（Processes）的核心概念 [1]，本设计对相关理论进行了如下分析：

1. 并发模型：接入层以非阻塞 I/O 处理连接，空间计算和存储交给工作单元；同房间写入仍由单一队列串行完成，不同房间可并行。浏览器保留地图渲染等本地计算，属于职责偏向展示的客户端，并非完全无计算的终端。2. 虚拟化：虚拟机或容器提供部署隔离，不能代替状态一致性。3. 代码迁移：下载前端脚本可联系课件的弱代码移动；本项目不迁移运行栈和执行上下文，不需要强移动性，也不把房间数据迁移误称为进程代码迁移。服务器集群通过接入层路由分配房间；TCP handoff 涉及连接移交，当前通过重连重新定位而不依赖该机制。[1]

4. 通信

4.1 技术需求

在虚拟桌游平台中，通信子系统负责节点间的信息交换，其面临如下技术需求与网络挑战：

（1）业务交互特征的分化需求：房间创建、配置读取等操作频率低、属于典型的单次交互；而拖拽标记、掷骰等交互频率高、要求低延迟的双向信息传递，通信体系需要支持差异化的协议分工。

（2）网络不可靠性应对需求：移动网络或公网环境下，消息可能面临传输延迟、意外丢包、报文乱序到达以及连接中断。通信协议必须考虑这些异常，不能假设消息一定会及时或成功到达。

（3）实时推送与历史恢复的分工需求：实时消息通道主要用于在线成员之间的快速状态扩散，而客户端掉线后的历史回溯则需要可靠的状态记录支撑，两者需合理衔接。

4.2 概要设计

(1) 请求—应答与实时协作的通信分工

系统将常规管理的请求—应答与实时协作的双向消息通道分开设计。[1] WebSocket 提供基于 TCP 的双向通信，可用于后者。[8] 不同业务场景下的通信方式对比如表 4-1 所示。

表 4-1 系统不同业务场景下的通信方式与协议特征对比表

业务交互场景	采用通信方式	通信交互模式	消息时效与持久性	设计理由与权衡
房间创建 / 加入验证	HTTPS / REST	请求—应答 (Request-Reply)	单次交互，服务端维护状态	语义明确，便于与常规认证和接入控制体系集成
预置地图授权瓦片查询	HTTPS / REST	请求—应答 (Request-Reply)	鉴权后裁剪；受限瓦片不进公共缓存	复用成熟的 Web 资源缓存机制，减轻实时通道负担
标记移动 / 掷骰指令	双向实时通道 (如 WebSocket)	双向全双工异步消息	即时发送，由服务端落盘确认	减少握手开销，提供较低的网络交互延迟
增量状态广播推送	双向实时通道 (如 WebSocket)	服务端逐成员单播，逻辑上组播	瞬态推送，历史依赖存储	及时向在线授权成员同步最新变化，避免频繁轮询
断线重新同步与追齐	双向通道 / REST 辅助	请求—应答式补发	从持久化记录读取历史更新	基于序号获取落后事件或拉取当前授权快照

(2) 角色标记移动与状态传播时序

系统在处理角色标记移动时的权威提交与状态传播时序如图 3 所示。

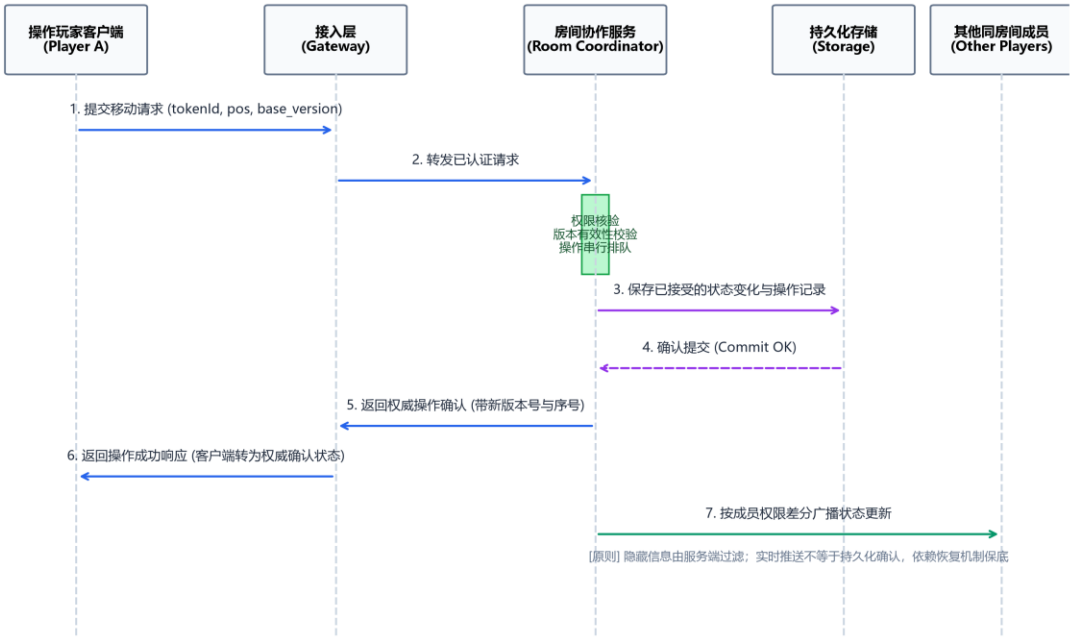


图 3　移动标记的权威提交与状态传播时序图

结合图 3，一次移动经过提交意图、接入路由、权威校验、同库事务确认和授权传播五步。请求携带标记标识、目标位置、base_version 及 operationId；协调服务检查身份、权限和版本，将状态、房间提交序号 roomSeq 与幂等结果原子保存，再返回确认。后台分发器扫描持久化事件，避免提交后进程崩溃造成永久漏通知。分发时比较每名成员操作前后的授权视图，形成新增、更新或移除事件；完全不可见的变化不下发。客户端依据成员专属视图序号 viewSeq 应用更新，roomSeq 仅表示权威历史位置。

(3) 课件理论分析：瞬态通信 vs 持久通信、消息广播与恢复边界

对照宋杰老师课件 Chapter 4（Communication）关于通信分类的论述 [1]，系统对消息机制进行了明确界定：

1. 瞬态通信（Transient Communication）的特性与局限：课件指出，瞬态通信仅在通信各方均正常运行时才能完成传递；若接收方正在经历断线或网络中断，消息往往被网络介质丢弃。WebSocket 属于典型的瞬态通信形式。系统明确界定：WebSocket 推送成功仅代表数据已写入发送缓冲区，绝不等于对端客户端已经确认接收，更不等于该数据已经持久化保存。

2. 持久通信（Persistent Communication）在恢复中的必要性：课件指出，持久通信在接收方未就绪时由存储中介保留消息。为了支持断线恢复，系统必须将已确认的关键状态变化写入持久化存储中保存。在线实时通道负责快速通知，而持久化存储记录则作为客户端断线后状态补齐的根本来源，二者互相配合形成完整的通信保障。

3. 广播与补发边界：房间广播是服务端逐成员单播形成的逻辑组播，不假定公网提供原子广播或可靠组播。[1, 15] WebSocket 所用 TCP 在同一有效连接上提供有序字节传送；应用仍要处理断连、进程崩溃和队列溢出。roomSeq 是房间提交序号，权限过滤会使其合法跳号，客户端不据此判断丢失；每个授权视图流另有连续 viewSeq，只有实际下发的可见事件占号。客户端对同一流的 viewSeq 去重、按序确认；缺口、超时或心跳报告的流进度落后触发补发。补发记录缺失时转为快照，绝不发送隐藏事件填号。

4. 通信模型的补充比较：RPC 通过客户端存根封装参数、服务端存根解包与调用、返回值反向传递，提供远程调用抽象，但远程调用仍有网络超时和部分故障；本设计管理接口采用 HTTP 请求—应答，不把 REST 与 RPC 概念等同。异步调用可避免等待通知阻塞，但不消除失败语义。消息队列可实现持久化中介；当前用数据库日志承接这一恢复职责，无需另设业务消息队列。课件面向流通信包含离散字节流及有时间约束的连续媒体；桌游使用离散状态消息，目标是应用延迟，不要求音视频抖动控制、流间同步或网络带宽 QoS。小房间适合服务端扇出，树形或流言传播更适合大规模群体，本阶段不采用。[1]

5. 命名

5.1 技术需求

在分布式协同环境中，实体识别与资源寻址是系统运转的基础，命名需求体现在以下层面：

（1）全局实体唯一标识需求：用户、房间、角色标记及底图资源需要稳定、跨节点唯一的标识符，便于在网络交互与存储中精确引用。

（2）便于用户分享的房间凭据需求：玩家加入房间需要输入人类友好、便于在即时通信工具中传播的简短邀请凭证，且该凭证需与底层持久房间标识解耦，并具备可撤销性。

（3）网络物理位置透明性需求：服务端节点可能随维护或扩容发生变化，客户端请求不能与具体的物理 IP 或端口强绑定，需要提供灵活的服务定位与路由机制。

5.2 概要设计

(1) 稳定实体标识设计

命名设计遵循稳定标识与网络位置分离的原则。[1, 2] 本系统选择 UUIDv4 作为业务实体的内部标识，并由权威库唯一约束拒绝重复值。核心实体包括用户标识（userId）、房间内部标识（roomId）、标记标识（tokenId）与地图资源标识（mapId）。UUIDv4 用于内部引用与寻址，其随机性不能替代授权；获取某个实体标识不等于获得操作权限，服务端仍须核验会话身份、成员角色和对象权限。

此外，系统引入操作意图标识（operationId），由客户端在构造写操作时生成并附带，用于服务端在网络超时重试时识别重复请求并辅助实现幂等性。

(2) 邀请码凭证与房间定位机制

为了兼顾分享便利性与安全性，系统将‘加入凭证’与‘房间持久标识’清晰解耦：（1）房间邀请码（Join Code）：生成易读短码（如 6~8 位字母与数字组合）。邀请码仅作为申请加入房间的临时校验凭证，不作为数据库持久主键；服务端对邀请码进行安全哈希存储，并支持主持人随时重置邀请码使旧码作废；（2）房间服务定位与路由：系统在路由缓存中维护房间标识至负责服务实例的映射关系。客户端发起实时协同连接时，接入层依据该路由映射将请求定向至负责该房间的协作服务节点。系统核心命名规则如表 5-1 所示。

表 5-1 系统核心实体命名与服务寻址规则表

命名对象分类	采用标识形式	有效作用域	寻址与解析方式	设计目的与说明
业务持久实体 (Room/User/Token)	全局唯一标识 (如 UUIDv4)	全生命周期有效	数据库主键索引直查	去中心化生成，具备位置无关性；仅为标识，不能替代服务端权限核验
房间社交加入凭证	6~8 位字母数字短码	房间配置期有效，支持重置	通过哈希校验解析至 roomId	便于人类记忆与社交分享，哈希存储防泄露
操作意图标识 (Operation)	客户端生成唯一 ID	幂等记录保留期内；绑定用户、房间及历史代次	权威库唯一约束及结果记录查询	用于识别超时重试请求，辅助实现幂等性
服务端网络物理地址	动态 IP:Port 组合	节点进程运行期有效	服务发现 / 路由目录查询	网络物理位置对客户端透明，便于节点动态调度
预置静态地图资源	结构化 URI 层次命名	静态资源长期有效	鉴权地图资源服务；公共缓存只放非敏感内容	便于分类管理与边缘缓存

(3) 课件理论分析：平面命名、结构化命名与 DHT/Chord 方案对比

对照宋杰老师课件 Chapter 5（Naming）的理论框架 [1]，系统对不同命名方案进行了分析：

1. 平面命名（Flat Naming）的适用性：课件指出平面标识符不包含位置信息。系统采用的 UUID 属于平面命名，其优势在于当房间在不同物理服务节点之间调度迁移时，其业务标识符无需发生变化，实现了良好的位置独立性。

2. 结构化命名（Structured Naming）在资源管理中的应用：课件介绍了树状层次化命名体系。系统在管理 API 与静态地图资源路径上采纳结构化 URI 命名（如 /api/v1/rooms/{roomId}/tokens），符合直观的逻辑从属关系，便于路由分发与分层管理。

3. 分布式哈希表（DHT / Chord）适用性权衡分析：课件 Chapter 5 详细讲授了 Chord 算法及其基于哈希环与指位表（Finger Table）进行去中心化寻址的机制 [1, 14]。

方案分析：DHT/Chord 适合于节点规模巨大、频繁动态加入与退出的完全对等系统。然而在雾桌平台的应用场景下，系统初始设计假设承载约 100 个活跃房间，服务节点均属于中心受控的云端服务实例。若在百级规模下强行引入 Chord 覆盖网络，为了定位房间往往需要跨节点进行多跳（Hop）网络查询，不仅显著增加了交互延迟，而且需要额外维护复杂的节点加入、退出与指位表周期探测逻辑。相较之下，采用集中式的内存路由目录能够以极低的时延（一次查询）完成寻址，运维与实现成本也显著更低。因此，系统在技术比较后明确判定当前阶段不采用 DHT/Chord，体现了以需求为导向的合理取舍。

4. 其他命名方式与解析：DNS 将服务域名解析为接入地址，接入层再以 roomId 查受控目录，形成层次化名称解析；转发指针易形成迁移链，家乡节点定位引入固定依赖，当前选择直接更新房间目录并重连定位。属性命名用于按地图标签、尺寸等条件检索，输出

mapId 后仍须鉴权，不以搜索结果代替授权；现阶段普通索引查询即可，无需部署 LDAP。DHT/Chord 适合开放对等查找，本系统受控节点规模下不采用。[1]

6. 同步

6.1 技术需求

在虚拟桌游的实时协同交互中，多人针对同一共享画板的并发操作不可避免，同步设计面临如下核心需求：

（1）空间标记并发修改的防静默覆盖需求：两名玩家可能在近似相同的微小时间窗口内试图移动同一个角色标记。系统不能允许后到达的网络请求无条件覆盖先达请求，必须提供可解释的冲突检测与状态仲裁机制。

（2）物理时钟不可靠下的排序需求：不同客户端设备的本地物理时钟存在几十毫秒至数秒不等的偏差，网络传输延迟亦不均衡。系统不能依赖客户端本地物理时间戳作为判断操作先后的权威依据。

（3）事件序列与因果关系的明确性需求：掷骰结果、角色位移与回合流转之间具有明确的因果关系，房间各成员端所呈现的事件演进历史应当具备一致的因果逻辑。

6.2 概要设计

(1) 基于房间权威定序与对象版本的冲突处理机制

为了在保证逻辑清晰的前提下实现并发控制，系统确立了‘单房间排队定序’与‘标记对象乐观版本检查（Version Check）’相结合的机制 [1, 2]。图 4 详细展示了两名玩家近似同时请求移动同一标记时的冲突检测与处理流程图。

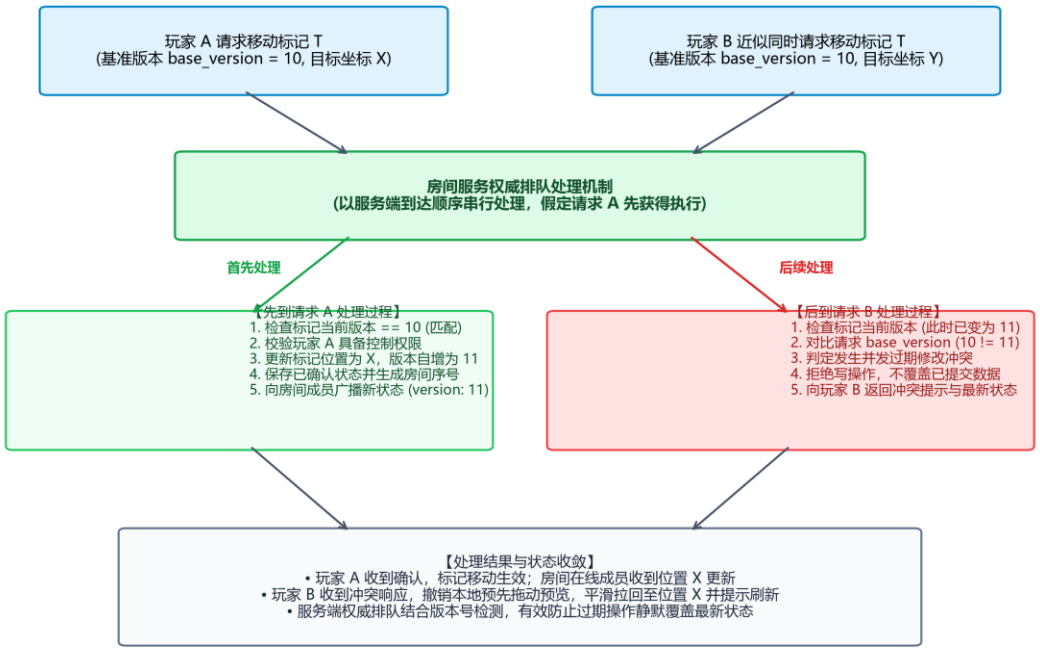


图 4　同一标记并发移动冲突与版本处理流程图

结合图 4，假设两名玩家均有权查看该标记：二者携带相同 base_version 请求移动，

协调者依队列处理，首个版本匹配的请求完成事务并递增对象版本；随后请求因版本落后被拒绝，返回其有权查看的当前状态。冲突方撤销非权威预览。不同客户端之间由服务端决定仲裁次序；同一客户端的写意图则只允许一个未决请求，后续请求须待前一请求结果明确后发出。超时先用同一 operationId 查询或重试，未决之前不发依赖它的新写；重连仍保留这一顺序。这样房间提交序列既统一定序，也保留每个客户端的程序顺序。权限不同的成员只应用该权威历史的授权投影，不要求所有成员收到完全相同的数据。

(2) 掷骰与状态判定的有序性保障

对于掷骰检定与回合流转，系统由服务端负责逻辑仲裁：客户端发送掷骰请求，服务端生成结果并在持久化存储中完成确认，赋予递增序号后向全房间广播。所有在线端依据接收到的事件流顺序记录投骰日志，确保房间成员观察到的投骰历史具备统一的逻辑全序先后关系。

(3) 课件理论分析：物理时钟偏差、逻辑时钟与并发控制算法比较

对照宋杰老师课件 Chapter 6（Synchronization）关于分布式时钟与互斥算法的理论 [1]，系统对相关方案展开了对比分析，如表 6-1 所示：

表 6-1 分布式时钟与并发控制算法适用性对比分析表

同步 / 互斥技术机制	核心运作原理	在虚拟桌游系统中的适用性分析	采纳结论与设计权衡
物理时钟同步 (NTP)	通过网络时间协议周期校准节点物理时钟	由于网络传输时延抖动，节点间存在数十毫秒偏差	仅作为日志记录与监控参考，不作为并发操作定序的权威依据
Lamport 逻辑时钟 [3]	满足因果先行则逻辑时钟递增；逆命题不成立	课件经典理论模型，用于解释分布式因果依赖与偏序概念	与本系统定序机制明确区分：系统采用服务端分配单调事件序号实现房间内全序，理论模型用于阐释因果依赖
向量时钟 (Vector Clock) [6]	各节点维护向量数组，精确捕获并发因果	课件高级理论。支持无中心对等节点间的并发分支捕获	在单房间集中仲裁模型下元数据开销偏大，非核心实现所必需
单房间权威协调者定序	单房间由单一权威路径顺序排队处理	在服务端入口消解并发冲突，逻辑清晰且开销确定	【核心方案】：以清晰的工程思路实现房间内的操作全序定序
Ricart-Agrawala 分布式互斥 [5]	基于组播与应答的去中心化临界区访问	进入临界区需发送 N-1 条请求消息并接收 N-1 条应答消息，共 2(N-1) 条消息；任一节点失联均易造成阻塞	仅作课件学术对比，在存在中心服务器的体系下消息开销过大，不采用
令牌环互斥 (Token Ring)	逻辑环上传递令牌，持牌节点方可操作	令牌丢失恢复协议复杂，且难以适应浏览器客户端随时退出的情况	仅作课件学术对比，不适合动态 Web 客户端环境，不采用

理论概念对应：Lamport 逻辑时钟 [3] 说明因果先行与时间戳的关系，向量时钟 [6] 用于识别因果与并发，Ricart-Agrawala 算法 [5] 解决消息传递网络中的互斥。以下将理论模型与本系统工程机制区分说明：

第一，Lamport 时钟遵循因果先行 a→b 蕴含 C(a)<C(b)，反向不成立，单凭标量时钟不能判定两个事件是否并发；向量时钟能够识别因果与并发关系。[1, 3, 6] 本系统的 roomSeq 是集中权威提交序号，viewSeq 是成员授权流序号，都不冒称 Lamport 时钟。核心操作依赖服务端已确认的状态与对象版本，不承诺未提交客户端意图之间的完整因果追踪。

第二，互斥算法的架构适用性权衡：Ricart-Agrawala 算法适用于无中心的对等网络，但每次进入临界区需全网组播 N-1 条请求消息并等待接收 N-1 条应答消息，进入临界区前的消息总数为 2(N-1) 条，且对节点故障极度敏感；而在本系统选定的分层 C/S 架构下，服务端房间协调服务充当了天然的逻辑仲裁点，通过请求排队结合乐观版本校验即可低开销化解并发写冲突，无需在客户端之间运行开销巨大的分布式互斥协议。应当指出，单房间串行化处理牺牲了一定程度的单房间多操作并行能力，但对于通常不超过 8 人的跑团房间而言，这种权衡能够极大简化逻辑并提升可靠性；全系统的扩展性则通过不同房间的并行处理来获得，二者不能混淆。

第三，协调者选举与唯一写入权是两个问题：课件 Bully 算法由发现故障者联系编号更高的进程，最高可用编号者成为协调者；环选举沿逻辑环传播候选者，再公布结果。[1] 当前受控集群通过数据库中房间归属记录的原子条件更新选择接管者，候选者先获得新任期再恢复写入，而不让浏览器运行 Bully 或环选举。超时只触发接管尝试，不证明旧进程已停止；存储逐次校验 fencing 任期才是阻止旧写入的安全边界，拿到最高编号本身不能代替该隔离。

7. 一致性与复制

7.1 技术需求

在多人协同交互平台中，不同业务数据对一致性强度的要求存在客观差异。系统需要根据业务特征实施分级的一致性设计：

（1）核心对局状态的确定性需求：标记、场景与掷骰结果形成单房间统一提交序列，保留各客户端已提交操作的程序顺序；各成员只观察与其权限相符的投影，不把网络到达时间视为全局因果证明。

（2）客户端交互的及时响应需求：用户在操作界面拖拽时，本地界面应能即时响应预览，不能因为等待网络往返而出现画面迟滞，需要支持客户端中心一致性体验。

（3）非关键瞬态信息的收敛需求：光标位置和输入提示允许短暂滞后，以最新采样覆盖旧状态；采样停止但成员仍在线时须重复发送最新状态直至收到确认，离线状态以会话失效清除，才能在网络恢复后收敛。

7.2 概要设计

(1) 业务分级一致性模型构建

对照课件 Chapter 7（Consistency and Replication）关于以数据为中心（Data-Centric）和以客户为中心（Client-Centric）的一致性模型分类 [1, 2]，系统构建了分级一致性策略，如表 7-1 所示。

表 7-1　系统各业务数据类型的一致性等级与保证策略表

数据业务分类	涉及典型实体	采纳一致性模型 / 交互机制	客户端感知体验	概要工程策略与保障前提
权威对局核心状态	标记位置、属性、迷雾区域、投骰记录	顺序一致性 (Sequential Consistency)	房间在线成员观察到全局一致的操作提交顺序	单房间串行提交且保留会话程序顺序；成员应用授权投影
本地操作交互体验	本端正在拖拽的标记、本端发起的移动	乐观交互预览 (Optimistic UI Preview)	拖拽即时视觉呈现，消除等待网络往返导致的交互迟滞	本地维护非权威推测状态，获服务端确认后固化；若冲突则拉回权威状态
客户端后续读取保证	本端后续查看房间、重连拉取快照	以客户为中心一致性 (Read-Your-Writes, Monotonic Reads)	同一历史代次内不退回已确认或已读水位之前	携带最大已读/已提交水位；不足时等待或拒绝；历史回退须显式重置
瞬态协同感知数据	成员鼠标光标位置、输入指示提示	最终一致性 (Eventual Consistency)	高频变动，短暂网络延迟下光标略有不同步，最终收敛	最新采样覆盖旧值；末次状态重发确认，离线用会话失效清除

理论概念对应与严格区分：

（1）顺序一致性要求操作结果可解释为同一串行序列，并保留每个进程的程序顺序。[1, 2] 系统以单房间串行事务和前述单未决写规则满足这一设计约束；读取来自该权威历史的已提交前缀，授权视图按序更新。前端推测状态不是已提交读取；网络延迟下不同成员可暂时处于不同前缀，系统不承诺所有端同时看到最新状态，也不把授权投影说成完整数据副本。

（2）乐观预览与客户端一致性：拖拽预览尚未提交，不能证明读自身写。确认后，客户端记录最大已提交及已读的权威水位，后续快照和读取不得低于它；存储尚未追齐时等待或返回暂不可读。[1, 12] 这一读自身写和单调读保证限于同一未回退的数据历史代次。若异步备库缺失已确认记录，单凭水位无法恢复数据，系统暂停该房间，等待补齐原日志；确实无法补齐时，须显式告知历史回退并经主持人确认后更换数据历史代次、重置全部视图。重置前不返回较旧结果冒充连续读取，旧代次的未决 operationId 不自动重放。

（3）其余模型及适用边界：单调写要求同一客户端后续写入基于已经包含其先前写入的副本；写跟随读要求写入所用状态不落后于该客户端最近读取。[1] 本设计以同一权威路径、客户端水位和对象版本检查约束这两项要求；条件不满足时拒绝或同步，不能让旧视图直接覆盖。因果一致性强调有因果关系的写入按序观察，独立并发写可不同序；本项目选择权威全序，不额外提供全 P2P 因果追踪。连续一致性以 Conit 为测量单位，分别限制数值、陈旧度和顺序偏差：角色位置及掷骰结果不接受任意数值偏差；光标可容忍滞后，但当前未为其量化三个上界，故不宣称实现了有界连续一致性。

(2) 战争迷雾与成员权限的状态投影隔离

在虚拟桌游平台中，一致性并不等于‘所有客户端存储完全相同的数据’。战争迷雾机制决定了不同角色对场景的可见范围存在客观不对称性。图 5 展示了基于角色与迷雾规则的视图状态投影原理。

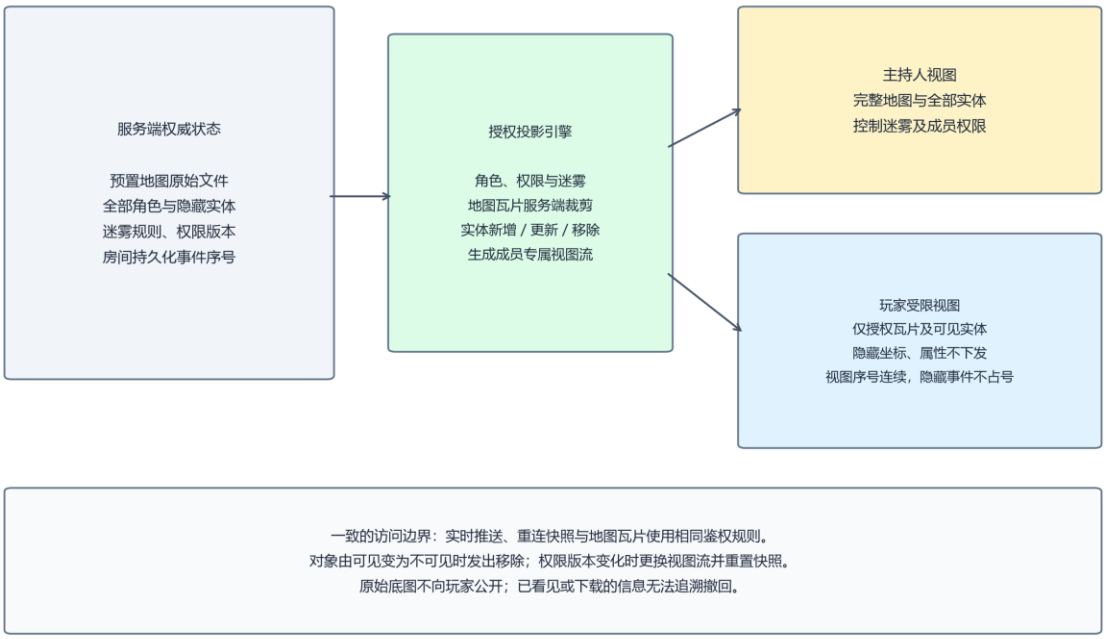


图 5　战争迷雾与角色权限状态投影图

结合图 5，服务端在实时事件、快照及地图瓦片上统一鉴权。预置底图原文件仅存服务端，玩家按地图标识、瓦片坐标及当前权限获取服务端裁剪后的图像；未探索像素及隐藏实体坐标、属性不进入玩家响应，公共 CDN 不保存受限瓦片。实体由不可见变为可见时下发当前授权状态，由可见变为不可见时发送只含已知标识的移除事件；双方均不可见的变化不占 viewSeq。成员权限或迷雾规则版本改变时终止旧视图流，发送重置通知，重新获取授权快照。已经披露的信息不能从玩家记忆或截图中撤回，撤权保证的是停止后续披露并清理当前视图，不作追溯保密承诺。

(3) 课件理论分析：主备复制与 Quorum 仲裁机制对比

对照宋杰老师课件 Chapter 7 关于多副本复制协议的理论 [1]，系统对主备复制与 Quorum 机制进行了对比论证：

1. 主备复制由主节点受理写入并传播日志，适合本系统单权威路径。若采用异步复制，主库已确认但尚未复制的记录可能在灾难故障后丢失，恢复遵循前述历史代次规则；若要求切换后保留全部已确认写，则确认条件必须包含所需副本的持久化及安全切换前提，不能只改一句“零丢失”。2. Quorum 比较：等权 N 副本条件 R+W>N、W>N/2 保证读写及写写集合相交；加权模型以总票数 V 替代 N，要求读写票数阈值满足 R+W>V、W>V/2。[1, 4] 交集条件还需版本选择、并发协调及故障假设，不能单独推出任意网络分区下的强一致。当前应用层不自建投票协议，交给成熟存储设施。

3. 副本管理与传播：公开静态资源适合 CDN/客户端缓存，核心状态读取以权威库为准，不任意从落后备库读取。课件区分永久副本、服务器发起及客户端发起副本，更新可推送、拉取或混合；本系统在线推送、恢复时拉取，缓存只服务资源访问，不越过权限和水位。主动复制要求多个节点按同序确定性执行，随机掷骰还需先统一随机结果；本项目选择主节点生成并记录结果的主备方式。本地写/迁移主节点可优化访问局部性，但增加归属转移复杂度，小房间当前不采用。[1]

8. 容错性与故障恢复

8.1 技术需求

在分布式网络环境下，局部故障是不可避免的常态。容错子系统必须具备应对以下典型异常的设计考量与恢复能力：

（1）弱网抖动下的状态对齐与恢复需求：玩家在无线网络或移动网络下面临短暂断连、丢包与延迟波动。系统需要设计状态判定与补发机制，在网络连通后使客户端尽快恢复到权威视图。需要强调的是，‘网络恢复后普通规模房间在 5 秒内恢复授权状态’属于未经压力测试的设计假设目标，用于指导差分恢复策略设计，而非经过实测验证的性能承诺。

（2）网络超时不确定性下的防重复操作与幂等需求：客户端提交掷骰或标记更新请求后，由于下行网络拥塞导致响应丢失，客户端在超时后可能发起重试。系统必须避免因网络重试导致操作被重复执行（例如重复扣减角色生命值或重复记录掷骰结果），该机制严格依赖持久化操作记录与本地事务边界。

（3）服务实例异常退出时的状态接管与防脑裂需求：负责特定房间的协作服务实例发生异常或失联时，备用接管实例需要从权威持久化存储拉取最新状态并恢复服务。在接管过程中，必须具备防止原实例与新实例同时写入的隔离机制（Fencing），避免产生双主（Split-Brain）数据混乱。

8.2 概要设计

(1) 网络超时分析与基于操作唯一标识的幂等重试机制

在分布式系统理论中，‘请求超时（Timeout）并不等同于操作执行失败’[1, 2]。当客户端发起掷骰请求且未在预期时间内收到确认响应时，实际物理网络中可能存在三种不同情况：① 请求报文在上传途中丢失，服务端未收到；② 服务端已成功接收并执行了本地事务提交，但在返回确认响应包时下行网络发生丢包；③ 服务端正处于高负载排队中，请求正在处理但尚未完成响应。

针对上述不确定性，客户端为每个写意图生成 operationId，重试复用原标识及原载荷。服务端先验证当前身份与历史代次，再以用户、房间、代次和 operationId 的唯一约束，将状态变更及原结果原子保存；命中已提交记录只返回原有授权结果，相同标识而载荷不同则拒绝，仍在处理则稍后重试。记录须覆盖允许的重试期限；超过保留期或发生历史代次重置时，不把旧意图当成新意图自动执行，须重新同步并明确结果。这样约束的是保留期内、同一持久历史的重复副作用，而非无条件的恰好一次。

(2) 客户端断线重连与增量/全量快照差分恢复流程

针对客户端断线重连，系统设计了差分状态恢复流程。图 6 展示了客户端断线重连与授权状态恢复的逻辑流转。

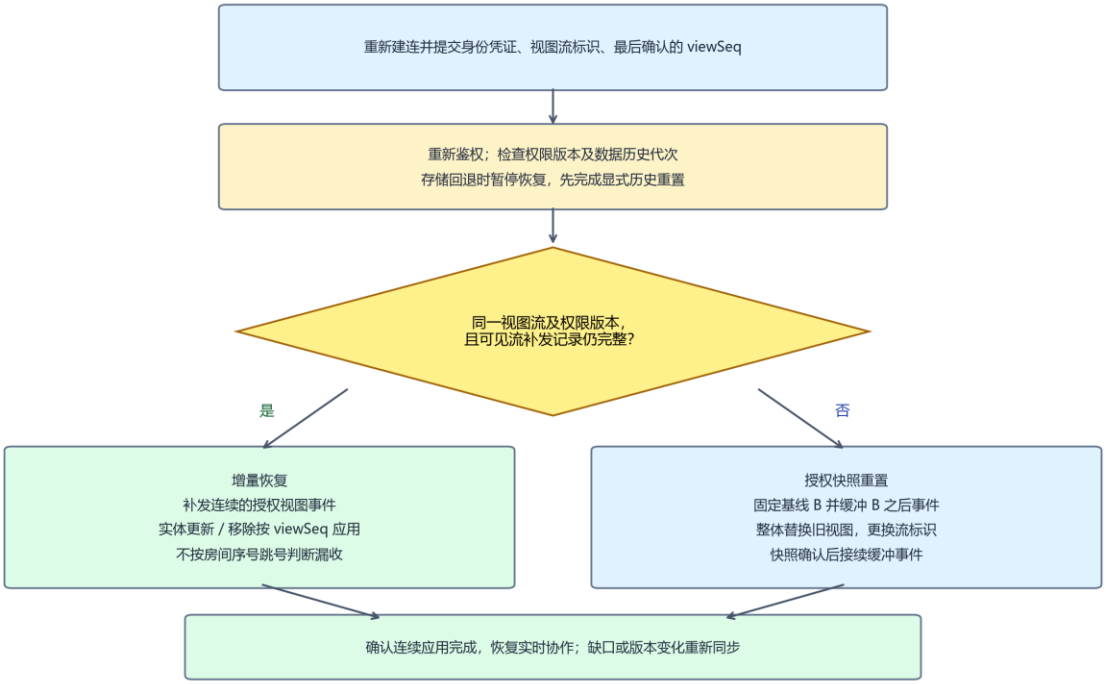


图 6　客户端断线重连与授权状态恢复流程图

结合图 6，重连先鉴权，客户端提交流标识、最后连续应用并确认的 viewSeq、权限版本和历史代次。同一授权流的补发记录完整时，服务端从确认点后补发可见事件；客户端拒绝旧流消息，对重复序号去重，对缺口暂停应用，不用 roomSeq 的合法跳号判漏收。授权视图流短期保留补发记录；服务实例重启而无法恢复原流、权限版本变化或记录超窗时，均改为快照重置。

快照重置先在权威库固定一致的提交基线 B，读取对应授权状态，同时缓冲 B 之后的已提交变化。客户端整体替换旧视图以清除失效对象，获得新流标识并确认基线后，再连续应用缓冲的可见事件；若缓冲溢出、权限再次改变或代次不符，废弃本次快照并重试，不能直接切到实时监听留下中间缺口。后台扫描持久日志并重试派发，心跳核对可见流进度，覆盖提交后未广播及末条消息遗漏。本机制是权威状态截面恢复，不是 Chandy–Lamport 去中心化全局快照算法；文献仅用于对照通道状态与截面概念。[1, 11] 五秒恢复仍是未经测试的设计目标。

表 8-1 典型网络与节点故障场景及恢复策略分析表

典型故障场景	根本故障成因与物理表现	系统预期容错与自愈策略	涉及课程理论与知识点
客户端网络突发闪断	无线网络信号抖动，长连接异常断开	客户端指数退避重连；服务端保留短期会话并走增量补发或快照重置	Chapter 4 瞬态通信缺陷；Chapter 8 崩溃恢复模型
请求提交成功但响应丢失	上行成功入库，下行网络拥塞引发客户端超时	客户端复用 operationId 重试，服务端命中幂等记录直接返回原结果	Chapter 1 事务边界；Chapter 8 RPC 幂等重试语义
房间协作服务实例异常退出	宿主机故障或协作进程崩溃	原子获取新任期并核验 fencing；从持久化状态恢复；旧流不可续则快照重置	Chapter 3 进程故障；Chapter 8 进程韧性与故障切换
实时事件应用层遗漏或连接中断	网关队列溢出、进程异常或断连导致未确认消息缺失	检查同一授权流 viewSeq/心跳进度；补发不足则授权快照重置	Chapter 4 成组通信；Chapter 7 顺序一致性与单调性
持久化存储短暂不可用	主数据库触发主备切换或瞬时磁盘 I/O 阻塞	协作层暂停该房间的新写事务，向前端返回降级重试提示	Chapter 1 ACID 持久性边界；Chapter 7 复制一致性

(3) 协作服务进程故障接管与恢复前提边界

依据课程的故障检测与恢复模型 [1, 2]，本设计在协调服务故障时仅将心跳超时视为“疑似失联”。候选实例在权威存储上原子争取该房间的新任期，成功者加载已提交状态后接管。每次写入都在同一事务内检查有效任期，拒绝旧实例；续租与到期判定采用受控服务端时间或一致性存储，不信任客户端时钟。租约有时钟及暂停假设，安全边界最终是存储的原子任期检查；权威存储不可达则暂停写入。该方案尚未通过自动接管或网络分区测试，不把超时、最高编号或路由修改本身当作唯一写入证明。

(4) 课件理论分析：故障分类模型、可靠通信与 2PC/3PC 适用边界

对照宋杰老师课件 Chapter 8（Fault Tolerance）关于容错理论与分布式提交协议的权威论述 [1]，系统建立如下理论映射：

1. 故障模型界定（Fault Models）：课件将分布式系统故障划分为崩溃故障（Crash-Stop）、崩溃恢复故障（Crash-Recovery）、省略故障（Omission Failure）、定时故障（Timing Failure）以及拜占庭故障（Byzantine Failure / Arbitrary Failure）。雾桌平台将系统容错边界明确界定在‘非拜占庭的崩溃-恢复故障与网络省略故障’范围内：假定服务端进程与数据库发生故障后会重启恢复，网络消息可能丢包或延迟，但服务端节点内部假定不存在伪造恶意数据的拜占庭叛徒行为。对于客户端可能存在的伪造请求或作弊嗅探，系统通过严格的服务端鉴权与战争迷雾差分投影加以防御，而非在系统内部引入开销巨大的拜占庭容错协议。

2. 可靠通信与 RPC 失败语义（RPC Semantics Under Failure）：课件分析了 RPC 面临节点崩溃或消息丢失时的常见语义：至少一次（At-least-once）、至多一次（At-most-once）与恰好一次（Exactly-once）。在不可靠的物理网络环境下，网络消息本身无法天然保证只投递一次。雾桌平台在网络层采用带有超时的重试机制（体现‘至少一次’通信倾向），在应用层结合唯一 operationId 幂等记录与事务更新，从而在端到端应用语义层面逼近‘恰好一次（Effectively-once / Idempotent Execution）’的处理效果，其本质是‘至少一次传输 + 幂等去

重’，而非物理网络保证消息只投递一次。

3. 分布式两阶段提交（2PC）与三阶段提交（3PC）适用性论证：课件 Chapter 8 及教材第 8.5 节介绍了 2PC 与 3PC 在保障跨节点事务原子提交方面的机制与限制。[1, 2]

设计论证：2PC 面向多个独立事务参与者的原子提交，参与者在准备后若无法获知决定可能阻塞。[1, 10] 3PC 增加预提交阶段，其非阻塞讨论依赖有界延迟等更强假设，网络分区或假设失效时不能据此保证安全且可用。[1, 2] 当前将房间状态、事件和幂等结果纳入同一权威库事务，数据库在正确持久化与隔离配置下承担 ACID，不涉及独立资源管理器的原子提交，故不自建 2PC/3PC。数据库复制及通知送达仍是另外的边界；单库事务不自动解决它们。

4. 进程韧性、可靠组通信与恢复：课件 k 容错讨论在故障假设下允许 k 个节点失效所需的冗余；崩溃故障与拜占庭故障的副本数量条件不同，本项目未固定副本部署，不宣称达到某个 k 容错等级。[1] 文献 [15] 讨论故障条件下可靠组通信及事件交付顺序；本系统采用不同授权视图流，不将其等同于虚拟同步协议。恢复时先回到已提交的检查点，再回放持久日志，属于检查点结合日志的后向恢复。[1, 2] 旧进程恢复后须重新核验任期，不以客户端画面回滚代替分布式故障恢复。

9. 总结

9.1 系统设计与分布式知识点对应回顾

本报告针对选题 B‘多人实时协作的在线虚拟桌游平台’，完成了一套面向课程要求的分布式系统概要设计说明书。系统紧扣分布式系统的核心特征与工程权衡展开设计，涵盖了《分布式系统导论》课程教学大纲与宋杰老师课件中 Chapter 1 至 Chapter 8 的核心理论模块[1]，尝试将理论概念与业务架构建立对应。课程核心知识点在系统设计中的应用与映射如表 9-1 所示。

表 9-1 课程 Chapter 1–8 知识点在系统设计中的应用与映射对应表

课程课件章节	核心概念与理论依据	本系统概要设计中的具体映射与落地	不采用或边界限定说明
Chapter 1 Introduction	分布式目标、透明性、陷阱、ACID 事务	定义网络延迟假定；以单库本地事务为边界；透明性与迷雾隔离权衡	不把数据库提交成功等同于网络广播送达
Chapter 2 Architectures	分层架构、C/S、P2P 对等网络、事件驱动	分层客户端—服务器架构；按房间分配权威协作；事件驱动异步通知	分析并排除全 P2P 架构（基于防作弊与迷雾要求）
Chapter 3 Processes	线程模型、并发处理、客户端分工、代码迁移	瘦客户端交互、服务端权威排队；异步事件驱动思路	论证桌游业务无需动态代码迁移，予以排除
Chapter 4 Communication	RPC 存根、瞬态/持久消息、离散/连续流、组播	HTTP 管理接口、双向消息、持久日志扫描和授权视图流	不把 REST 当作 RPC;无音视频，不引入媒体 QoS 和大规模组播
Chapter 5 Naming	平面/结构化/属性命名、DHT、目录解析	稳定 ID、邀请码、受控路由、地图标签检索及鉴权瓦片	比较转发指针、家乡节点与 Chord；当前无需 LDAP 或覆盖网络
Chapter 6 Synchronization	物理/逻辑时钟、向量时钟、互斥、Bully/环选举	单房间定序、会话程序顺序、版本检测与原子任期接管	房间序号不等于 Lamport 时钟；选举不等于 fencing
Chapter 7 Consistency	顺序/因果/最终/连续一致性、四类客户端模型、复制	授权投影、视图序号、水位、快照衔接与历史回退边界	Conit 不承诺未量化界限；异步回退暂停并显式更换历史代次
Chapter 8 Fault Tolerance	故障模型、k 容错、可靠组通信、日志恢复、2PC/3PC	保留期内幂等、快照/增量分流、存储任期隔离	不宣称拜占庭或虚拟同步实现；单库不自建跨库提交

9.2 核心设计取舍与工程理性总结

本设计将课程知识落实到并发移动、掷骰重试和迷雾隔离：C/S 统一校验，单房间定序与对象版本处理冲突，房间间并行支撑扩展。授权视图流区分隐藏事件与漏收，实体移除及快照维护正确视图，受控瓦片限制底图披露。对 CRDT、P2P、应用层 Quorum 和跨库提交，均依据当前需求说明不采用的理由。[1, 13]

9.3 设计局限性与未验证假设声明

本报告是概要设计，未完成原型、压测或故障注入。每房间最多 8 人、约 100 个活跃房间、实时 P95≤300 ms、API P95≤500 ms、恢复≤5 秒均是设计目标。异步复制丢失已确认历史时须暂停并显式恢复，跨历史代次不承诺连续读自身写或去重；已披露信息不能撤回。后续须验证授权瓦片、视图流、fencing 和快照衔接，并评估单房间串行瓶颈，不能把当前推导当作已实现保证。

参考文献

[1] 宋杰（修改）. 分布式系统导论（Introduction to Distributed System）[Z]. 东北大学软件学院教学课件, 2014（据课件页脚修改年份著录）.

[2] TANENBAUM A S, VAN STEEN M. Distributed systems: principles and paradigms [M]. 2nd ed. Upper Saddle River: Pearson Prentice Hall, 2007.

[3] LAMPORT L. Time, clocks, and the ordering of events in a distributed system [J]. Communications of the ACM, 1978, 21(7): 558-565.

[4] GIFFORD D K. Weighted voting for replicated data [C]//Proceedings of the seventh ACM symposium on Operating systems principles. 1979: 150-162.

[5] RICART G, AGRAWALA A K. An optimal algorithm for mutual exclusion in computer networks [J]. Communications of the ACM, 1981, 24(1): 9-17.

[6] MATTERN F. Virtual time and global states of distributed systems [C]//Proceedings of the Workshop on Parallel and Distributed Algorithms. North-Holland, 1989: 215-226.

[7] FIELDING R T. Architectural styles and the design of network-based software architectures [D]. Irvine: University of California, Irvine, 2000.

[8] FETTE I, MELNIKOV A. The WebSocket Protocol [R]. RFC 6455, 2011.

[9] Owlbear Rodeo. Owlbear Rodeo [EB/OL]. [2026-10-24]. https://www.owlbear.rodeo/.

[10] GRAY J. Notes on data base operating systems [M]//Operating Systems: An Advanced Course. Lecture Notes in Computer Science, vol. 60. Berlin: Springer, 1978: 393-481.

[11] CHANDY K M, LAMPORT L. Distributed snapshots: determining global states of distributed systems [J]. ACM Transactions on Computer Systems, 1985, 3(1): 63-75.

[12] VOGELS W. Eventually consistent [J]. Communications of the ACM, 2009, 52(1): 40-44.

[13] SHAPIRO M, PREGUIÇA N, BAQUERO C, et al. Conflict-free replicated data types [C]//Stabilization, Safety, and Security of Distributed Systems. SSS 2011. Lecture Notes in Computer Science, vol. 6976. Berlin: Springer, 2011: 386-400.

[14] STOICA I, MORRIS R, KARGER D, et al. Chord: A scalable peer-to-peer lookup service for internet applications [J]. ACM SIGCOMM Computer Communication Review, 2001, 31(4): 149-160.

[15] BIRMAN K P, JOSEPH T A. Reliable communication in the presence of failures [J]. ACM Transactions on Computer Systems, 1987, 5(1): 47-76.

附录 (得分栏勿填)

组名	系统名	组长			副组长 1			副组长 2			副组长 3			组员 1			组员 2			组员 3			组员 4		
		姓名	学号	得分	姓名	学号	得分	姓名	学号	得分	姓名	学号	得分	姓名	学号	得分	姓名	学号	得分	姓名	学号	得分	姓名	学号	得分
完成作业组	雾桌	周宇斌	2024 6085		刘润七	2024 6162		刘万轩	2024 6063		周星宇	2024 5986													